

Git Situations

Backup plan for messed up Rebase

1. Make a copy of the current branch (e.g `git branch my_branch_backup`)
2. In case of bad rebase, `git reset my_branch_backup --hard`

Undo a Merge

1. `git reset --merge ORIG_HEAD`
2. Merge commit will be garbage collected later.
3. `--merge` flag to preserve uncommitted changes.

Use rebase to split commits

1. Start an interactive rebase with `git rebase -i`
2. mark the commit with an `edit`
3. `git reset --mixed HEAD^`
4. `git add`
5. `git commit`
6. repeat (4) & (5) until the working area is clean!
7. `git rebase --continue`

Amend any arbitrary commit

1. `git add` new files.
2. `git commit --fixup <commit>` . This creates a new commit, the message starts with *fixup!*
3. `git rebase -i --autosquash <commit>^`
4. git will generate the right todos. Just save and quit.

Basic Remote Workflow

1. Clone repo `git clone <URL>` if you haven't cloned already.
2. Fetch remote

3. Make changes
4. Fetch remote (steps 7 to 9)
5. Stage changes `git add <file>`
6. Commit to local `git commit -m "add new file"`
7. Fetch origin `git fetch origin`
8. Merge/Rebase origin if needed `git merge origin/master`
9. Or pull directly instead of steps 7 & 8 `git pull origin ( --rebase )`
10. Push to remote (origin) `git push origin master`

Clone forked repo and add upstream

1. Clone fork `git clone <fork_URL>`
2. Add upstream `git remote add upstream <upstream_URL>`
3. Fetch upstream when needed `git fetch upstream`
4. To merge upstream/master with local master `git merge upstream/master`

Move commits without Merge

1. Check out the branch where we would like to cherry-pick.
2. `git cherry-pick <commit>`
3. Done.

Find bad commits that introduced a bug

1. Start a bisect `git bisect start <BAD_SHA> <GOOD_SHA>` . Git bisect checks out in binary search mode, so the chance of finding a bad commit is much quicker.
2. Perform a manual test, for passed test case run `git bisect good` , and for failed test case run `git bisect bad` .
3. Or, run a script that returns `SUCCESS` (exit code 0) or `FAIL` . `git bisect run <command> <args>`

File change history

```
git log --name-status --follow --oneline hello.template
```

Recover a deleted file

1. Find in which commit the file was deleted `git log --diff-filter=D --oneline --<file>`
2. Check out the file from the previous commit of deleting. `git checkout <commit>^ -- <file>`

Delete commits/update a branch to a specific commit

1. Checkout to that branch
2. Hard reset to that commit

```
git reset --hard <commit>
```

or

```
git reset --hard HEAD~n
```

3. Now make new changes from there. Other commits will be counted as Dangling Commit and will be garbage collected.
4. To push to remote, do a force push (make sure there is no branch protection)

```
git push origin HEAD --force
```

Delete a file from an arbitrary commit and update whole history

-

দুই ভাবে করা যাবে -

১. কনফিগ রিমুভ করে নতুন করে পুশ (সেইফ ফর গিটহাব)*
 ২. যেই কমিটে কনফিগ এড হইসে, সেটাকে ফিক্সআপ করে ইন্টেরেক্টিভ রিবেইজ করা।
- ১ নং উপায়ে করলে গিটহাবের হিস্টরি রি-রাইট করবে না। কিন্তু এতে পুরানো কমিটগুলোতে যদি এই ফাইল থাকে, তাহলে সেটাকে দেখা যাবে হিস্টরি থেকে।

নরমালি রিমুভ করে পুশ করতে পারেন অথবা কমান্ডের মাধ্যমে করতে পারেন -

১. `git rm --cached /path/to/your/config`
২. `git commit -am "remove config file"`
৩. `git push origin main`

কিন্তু আপনি যদি চান একেবারে সব কমিট থেকে রিমুভ করতে, তাহলে ফিক্সআপ রিবেইজ করা লাগবে। এতে একটাই সমস্যা, গিটহাবে নতুন করে হিস্টরি রি-রাইট হবে। ঐ রিপো তে একসাথে কয়েকজন কলাব করলে এই উপায়টা recommended না।

২নং এর জন্য প্রথমে জানতে হবে আপনার কোন কমিটে এই কনফিগ ফাইল এড হইসে। সব থেকে ইজি ওয়ে হচ্ছে লগ কমান্ডের মাধ্যমে খুজা -

```
git log --oneline --diff-filter=A --follow -- path/to/your/config
```

এই কমান্ড রান করলে কোন কমিটগুলোতে এই ফাইলে কিছু এড হইসে দেখা যাবে। একেবারে নিচের কমিটের হ্যাশ কে কপি করে রাখেন।

তারপর আপনার কনফিগ ফাইল ডিলেট করেন, স্টাজিং এ এড করেন (`git add .`)।

`git status` দিয়ে চেক করে নিতে পারেন Changes to be committed: এ আপনার কনফিগ ফাইল delete এর লাইন আছে কিনা।

এবার একটু ডিফারেন্ট way তে কমিট করবো। এটা আমাদের ফিক্সআপ কমিট, কমিট করার পর লগ দেখলে দেখবেন যে fixup! আছে কমিট ম্যাসেজের শুরুতে।

```
git commit --fixup <commit_hash_you_copied>
```

তারপর ইন্টেরেক্টিব রিবেইজ শুরু করতে হবে-

```
git rebase -i --autosquash <commit_hash_you_copied>^
```

[--autosquash ফ্ল্যাগের কারনে অটোমেটিক রাইট টুডু গিট করে দিবে।]

রিবেজ শুরু হলে সিমপ্লি এক্সেপ্ট করে নিলেই হবে (মানে রিবেইজ টুডু ফাইলটা ক্লোজ করে দিলেই হবে)। গিট আপনার প্রেফার্ড এডিটরে ফাইলটা খুলবে। vscode এ হলে Start Rebase বাটন দেখতে পাবেন, ওটায় ক্লিক করলেই হবে।

সাকসেফুলি রিবেইজ হলে এবার গিটহাবে পুশের পালা। নরমালি পুশ করতে গেলে ইরোর দিবে, কারন আপনার লোকাল হিস্টরি এর সাথে গিটহাবের হিস্টরির মিল নেই। সো আমাদের ফোর্স পুশ করতে হবে, যেটা আপনার গিটহাবের হিস্টরি রিরাইট করে দিবে।

```
git push origin main -f
```

ব্যাস, হয়ে গেলো হিস্টরি রিরাইট। পুরো স্টেপের কমান্ড একসাথে -

```
git log --oneline --diff-filter=A --follow -- path/to/your/config
```

```
git add . (after removing your config file)
```

```
git commit --fixup b5bd7e8
```

```
git rebase -i --autosquash b5bd7e8^
```

```
git push origin main -f
```

আর এই ফিক্সআপ রিবেইজ করা লাগবে না যদি লাস্টের কমিটে কনফিগ এড করা হয়। তাহলে রিমুভ করে, এড করে লাস্ট কমিট এমেন্ড (`git commit --amend`) করে দিয়ে ফোর্স পুশ করে দিলে হবে।

সব থেকে বেস্ট প্রাক্টিস হচ্ছে সিক্রেট ফাইল গুলো `.gitignore` এ লিস্ট করে ফেলা। হিস্টরি রিরাইট করা বেস্ট প্রাক্টিস না কোলাবরেশনে।

যেভাবে vscode কে ডিফল্ট এডিটর হিসেবে সেট করবেন

- <https://stackoverflow.com/a/33371263/14685101>